Methodological Support Report

Frontend and Backend Architectural Documentation

ANE Spectral Sensing Platform

Document type:	Technical-methodological support report
Scope:	Web frontend and backend platform layers
Evidence date:	Repository snapshot analyzed on May 22, 2026
Evidence sources:	`context/`, `frontend/`, `backend/`
Excluded from direct scope:	`worksheet-plan/`, deep RF DSP internals, `postprocesamiento/` implementa

Prepared through architectural recovery, static code inspection, and documentary triangulation

Contents

1 Methodological Framework

1.1 Methodological Position

This report adopts a descriptive and reconstructive software engineering methodology. It is descriptive because it documents the system as implemented rather than as hypothetically desired. It is reconstructive because the architecture is inferred from multiple evidence sources, including source code, route definitions, persistence structures, and contextual documentation. In software architecture terms, the report follows a concern-oriented viewpoint approach: stakeholders need to understand authentication, control, telemetry, persistence, reporting, and maintainability; the document therefore groups evidence according to those concerns.

1.2 Evidence Base

The report was produced from the following evidence classes:

Evidence class	**Representative sources**	**Methodological contribution**
Platform context	`context/CURRENT_PLAT.md`, `context/DIAGRAM-FRONTEND.md`, `context/DIAGRAM-BACKEND.md`, `context/API-DOCUMENTATION-BACKEND.md`	High-level functional framing, declared responsibilities, documented flows, and endpoint taxonomy.
Operational boundary context	`context/EDGE_NODE_PROCESSING.md`, `context/PLAT_PROCESSING.md`	Clarification of how frontend/backend interact with edge sensing and the Python reporting service, without entering full implementation scope.
Database context	`context/db/README.md`, `context/db/tables.md`, `context/db/constraints.md`, `context/db/hypertables.md`, `context/db/diagram.md`	Confirmation of relational entities, constraints, time-series usage, and evidence of schema evolution.
Frontend implementation	`frontend/src/main.tsx`, `frontend/src/App.tsx`, `frontend/src/contexts/AuthContext.tsx`, `frontend/src/services/api.ts`, selected components and hooks	Recovery of the client-side control model, rendering shell, data acquisition logic, and user workflow orchestration.

Evidence class	Representative sources	Methodological contribution
Backend implementation	`backend/src/app.ts`, route modules, middleware, models, database connection and migration files, WebSocket services	Recovery of the service topology, control logic, persistence behavior, authentication, and orchestration strategies.

1.3 Analytical Procedure

The analytical procedure followed four successive stages:

1. **Documentary triangulation**: the contextual documents were read to identify declared responsibilities, subsystem boundaries, and terminology.
2. **Static code inspection**: the active entry points, hooks, route files, middleware, models, and migration scripts were inspected to verify how the documented responsibilities are implemented.
3. **Architectural synthesis**: code modules were grouped into coherent architectural capabilities such as authentication, telemetry ingestion, monitoring control, campaign orchestration, and reporting.
4. **Methodological justification**: strategies, tools, and algorithms were assessed according to quality attributes such as traceability, temporal consistency, maintainability, security, and operational fit.

1.4 Evaluation Criteria

The report evaluates architectural choices through the following quality criteria:

- **Functional adequacy**: whether modules satisfy their declared operational responsibilities.
- **Temporal coherence**: whether the design handles time-sensitive flows such as real-time monitoring, sensor status decay, and scheduled campaigns.
- **Traceability**: whether responsibilities can be mapped to identifiable modules and routes.
- **Maintainability**: whether the decomposition favors controlled change, localized reasoning, and pragmatic evolution.
- **Operational robustness**: whether the design tolerates stale data, disconnections, or long-running tasks.
- **Security posture**: whether identity, session, and role concerns are explicitly handled at meaningful architectural boundaries.

2 System Foundation and Architectural Context

2.1 Domain Mission

The platform mission is to connect human operators, RF sensors, persistent storage, and analytical services in order to convert raw spectrum measurements into operational monitoring and compliance-oriented knowledge. In implementation terms, the platform is an integration system rather than a single-purpose data viewer: it must coordinate control, ingestion, visualization, and interpretation.

2.2 Subsystem Boundary

The repository reveals a four-layer operational context:

- **Edge sensor layer**: external field devices that emit status, GPS, spectrum, and audio, and that consume active configurations and campaign assignments.
- **Frontend layer**: React-based user interface for authentication, dashboarding, monitoring, campaigns, alerts, and administration.
- **Backend layer**: Express and WebSocket services that coordinate persistence, configuration, orchestration, and reporting.
- **Analytical extension layer**: external services for geolocation and Python post-processing used by the reporting workflow.

2.3 Core Architectural Principle

The dominant architectural principle is *control-plane and data-plane separation.* The backend does not only persist data; it also acts as the authoritative control plane for sensor configuration, campaign state, and status transitions. Meanwhile, the frontend focuses on orchestration of user workflows and high-temporal-resolution visualization rather than on domain ownership. This separation is methodologically justified because sensing systems require a stable backend source of truth when browsers disconnect, sessions expire, or multiple users interact concurrently.

2.4 Cross-Cutting Data Model

The database context confirms a domain model centered on sensors, antennas, campaigns, time-series telemetry, alerts, users, and cached compliance reports. Three design facts are particularly important:

- sensor identity is anchored by the sensor MAC address, which appears as the operational join key across telemetry and control tables;
- campaign configuration combines structured columns with JSONB configuration, enabling both normalized querying and flexible sensor-specific payload enrichment; and
- the persistence model is evolutionary: the legacy context confirms `sensor_status` as a TimescaleDB hypertable, while the current migration script attempts to optimize `sensor_data` as a hypertable when TimescaleDB is available.

3 Frontend Methodological Analysis

3.1 Frontend Scope

The frontend is responsible for the human-facing interaction model of the platform. Its scope includes authentication initiation, application routing, dashboard presentation, sensor selection, monitoring parameterization, live spectrum visualization, campaign CRUD interaction, report consultation, alert review, and selected administrative tasks.

3.2 Technical Approach

The frontend follows a component-based single-page application approach implemented with React 19, TypeScript, Vite, Axios, Leaflet, Plotly, and TailwindCSS. Methodologically, this is appropriate because the interface must mix control widgets, long-lived authenticated state, high-frequency chart refresh, and geospatial views. The approach favors local composability while keeping the network and authentication concerns centralized.

3.3 Architectural Structure

The client architecture is organized into five coherent strata:

1. **Bootstrap and routing**: `frontend/src/main.tsx` initializes MSAL, processes Azure redirect responses, exchanges the resulting token with the backend, and defines public/protected routes.
2. **Session context**: `frontend/src/contexts/AuthContext.tsx` centralizes application JWT storage, user loading, Axios token injection, and logout behavior.
3. **Application shell**: `frontend/src/App.tsx` acts as the orchestration hub for tabs, selected sensor state, monitoring lifecycle, global statistics, and the coupling between configuration and analysis panels.
4. **Service and data hooks**: `frontend/src/services/api.ts` and `frontend/src/hooks/useSpectrumData.ts` encapsulate HTTP access, polling cadence, stale-response protection, and conversion of raw backend payloads into chart-ready structures.
5. **Feature components**: specialized components implement monitoring, campaigns, alerts, configuration, audio, and network visualization.

3.4 Frontend Module Traceability

Module	Primary concern	Methodological role
`frontend/src/main.tsx`	Bootstrap and route entry	Establishes the authentication-first execution order: MSAL initialization, redirect handling, backend token exchange, and route mounting.
`frontend/src/contexts/AuthContext.tsx`	Session governance	Maintains the application-level JWT, resolves the current user through `/auth/me`, and coordinates protected UX behavior.

Module	Primary concern	Methodological role
`frontend / src / services/api.ts`	Service abstraction	Encapsulates REST access, base URL policy, timeout configuration, and typed interfaces for sensors, antennas, telemetry, alerts, and system configuration.
`frontend / src / App . tsx`	Application orchestration	Coordinates tab navigation, selected sensor context, monitoring lifecycle, statistics refresh, real-time WebSocket subscriptions, and waterfall memory.
`frontend / src / hooks / useSpectrumData.ts`	Live spectrum acquisition	Implements polling with abortable requests, stale-response rejection, reset semantics, and spectrum-to-chart conversion.
`frontend / src / components / ConfigurationPanel. tsx`	Monitoring control	Encodes the measurement parameterization workflow, antenna selection, presets, demodulation options, filter ranges, monitoring start/stop, and campaign prefill export.
`frontend / src / components / AnalysisPanel.tsx`	Spectral interpretation	Aggregates spectrum charting, waterfall rendering, marker management, export functions, unit conversion, and live audio control.
`frontend / src / components / MonitoringNetwork . tsx`	Device situational awareness	Combines inventory retrieval, real-time status/GPS updates, antenna assignment, and map-centered inspection of sensors.
`frontend / src / components / CampaignsList.tsx`	Campaign workflow	Supports campaign listing, filtering, pagination, modal launch, cancellation, deletion, and transition into campaign data viewing.
`frontend / src / components / CampaignDataViewer. tsx`	Historical replay and reporting	Binds campaign data loading, playback, chart review, and report generation around a selected campaign-sensor pair.
`frontend / src / components / ComplianceReport . tsx`	Report presentation	Renders compliance outputs, handles sensor-scoped generation, and supports JSON export for analytical traceability.
`frontend / src / components / AlertsPanel.tsx`	Operational alert fusion	Merges current sensor health evaluation with historical alert retrieval, thereby providing both present-state and retrospective views.

Module	Primary concern	Methodological role
`frontend / src / components / WebRTCAudioPlayer . tsx`	Real-time audio stream	Implements the browser-side listener for sensor audio streams and couples monitoring with audible interpretation.

3.5 Frontend Design Choices and Their Justification

3.5.1 Authentication Bootstrap Before Rendering

The frontend intentionally handles Azure redirect resolution before rendering the full application shell. This prevents transient inconsistent states in which the user interface appears unauthenticated while a redirect flow is still being resolved. Methodologically, this strategy improves session determinism and reduces race conditions between identity acquisition and route evaluation.

3.5.2 Centralized Auth Context

The use of a single authentication context is justified by the need for consistent authorization headers, session expiry handling, and UI-level authorization checks. Without such centralization, different features would duplicate token retrieval logic and degrade maintainability.

3.5.3 Hybrid Push-Pull Real-Time Model

The monitoring experience uses a hybrid model in which WebSocket traffic is leveraged primarily for event and audio delivery, while spectrum traces are refreshed by aggressive polling from the REST API. This design may appear redundant, yet it is operationally coherent: audio and control events benefit from push semantics, whereas dense spectral arrays are retrieved through a cache-aware endpoint that the frontend can poll at a controlled cadence. This lowers coupling between visualization refresh and the broader event bus.

3.5.4 Single Orchestration Shell

`App.tsx` is large, but it also acts as the explicit coordination locus for selected sensor state, monitoring flags, polling hooks, and tab-level workflow transitions. Methodologically, this expresses a deliberate central orchestration strategy: the platform privileges shared operational context over highly fragmented local state stores. This improves short-term traceability, even if future refactoring could subdivide orchestration responsibilities.

3.5.5 Feature-Oriented Componentization

The feature components are organized by operational concern rather than by visual primitive. For this platform, that is the correct abstraction level. Users think in terms of monitoring, campaigns, alerts, and devices, not generic cards or widgets. The component map therefore aligns well with domain workflows.

3.6 Frontend Strategy Justification

Strategy	Observed implementation	Justification
Authentication-first bootstrap	Azure redirect is processed before route rendering in `main.tsx`.	Avoids inconsistent protected-route evaluation and provides a deterministic session entry point.
Hybrid push-pull telemetry delivery	Spectrum is polled; status, GPS, control events, and audio use WebSocket paths.	Aligns transport mechanism with payload behavior: high-frequency scalar/event data benefits from push, while spectrum retrieval stays cacheable and queryable.
Centralized shell orchestration	`App.tsx` owns tab state, selected sensor, monitoring lifecycle, and chart-waterfall coupling.	Preserves operational coherence across screens that share the same sensor and acquisition context.
Progressive configuration-to-campaign workflow	Monitoring configuration can be reused to prefill campaign creation.	Reduces operator friction and preserves parameter continuity between exploratory and scheduled measurement.
Operational alert fusion	`AlertsPanel.tsx` combines present-state checks with historical alerts.	Produces a richer operational picture than history-only or snapshot-only dashboards.

3.7 Frontend Tool Justification

Tool	Observed usage	Justification
React 19	Component model, stateful orchestration, protected shell	Suitable for long-lived interactive dashboards with rapidly changing local state and reusable domain-specific UI units.
TypeScript	Typed service interfaces and component props	Reduces ambiguity in telemetry payloads, campaign objects, authentication state, and configuration envelopes.
Vite	Frontend build and development tooling	Supports a lightweight development loop and modern module semantics suitable for SPA delivery.
Axios	Central HTTP access layer	Simplifies token injection, long-timeout handling, and uniform REST interaction across features.
MSAL (Azure)	Browser authentication bootstrap	Appropriate for institutional identity federation and redirect-based sign-in.
Leaflet / React Leaflet	Sensor and campaign geospatial visualization	Methodologically aligned with map-centric monitoring and location-dependent reporting workflows.

Tool	Observed usage	Justification
Plotly	Spectrum visualization and export operations	Provides high-density charting features, image export, and flexible overlays appropriate for spectral analysis tasks.
TailwindCSS	Rapid UI styling	Enables consistent visual composition across complex operational screens without a heavy component framework.

3.8 Frontend Algorithm Justification

Algorithm or rule	Where it appears	Justification
Abortable polling with stale-response rejection	`useSpectrumData.ts`	Prevents obsolete responses from contaminating the currently selected sensor view; essential when operators switch devices during active polling.
Waterfall reset with grace window and signature deduplication	`App.tsx`	Ensures that frequency-range changes do not mix old and new traces and avoids duplicate waterfall rows when the spectrum has not materially changed.
Nearest-bin marker evaluation	`AnalysisPanel.tsx`	A pragmatic algorithm for spectral cursor placement when chart points are discrete and uniformly ordered.
Power-unit conversion	`AnalysisPanel.tsx`	Supports instrument-like interpretation by translating dBm values into voltage-derived units required by operators.
Client-side campaign pagination and filtering	`CampaignsList.tsx`	Adequate for moderate list sizes and improves usability without requiring a more complex server-side query layer.
Combined live-plus-historical sensor issue detection	`AlertsPanel.tsx`	Methodologically valid for monitoring because a present clean state does not erase earlier incidents, and historical alerts alone do not convey current operational risk.

4 Backend Methodological Analysis

4.1 Backend Scope

The backend is the authoritative coordination layer of the platform. It exposes management and ingestion APIs, persists operational data, validates identity, broadcasts real-time events, maintains system configuration, schedules campaign transitions, and orchestrates compliance reporting by interacting with external analytical services.

4.2 Technical Approach

The backend uses Node.js, TypeScript, Express, PostgreSQL, WebSocket services, JWT-based session authorization, Azure token validation through JOSE, and Swagger documentation. The approach is not microservice-heavy inside the platform core; instead, it prefers a concentrated integration backend that delegates only specialized analytical responsibilities outward. This is methodologically coherent for a platform whose primary challenge is coordination rather than isolated computational specialization.

4.3 Service Architecture

The backend architecture can be described as a route-model-orchestration style with explicit cross-cutting middleware:

1. **Server bootstrap**: `backend/src/app.ts` initializes Express, database startup, Swagger, the main WebSocket service, the audio WebSocket service, and periodic validation of sensor status.
2. **Middleware layer**: `middleware/auth.ts` handles local JWT validation and role checks; `middleware/azureAuth.ts` verifies Azure-issued JWTs through remote JWKS.
3. **Route layer**: specialized route files handle authentication, sensor ingestion/control, management CRUD, campaigns, reports, and system configuration.
4. **Model and persistence layer**: models encapsulate database-facing rules for sensors, telemetry, alerts, and active configurations.
5. **Real-time transport layer**: `websocket.ts` distributes general events; `audioServer.ts` handles dedicated audio ingest/listen paths and Opus-to-PCM decoding.

4.4 Backend Module Traceability

Module	Primary concern	Methodological role
`backend/src/app.ts`	Runtime composition	Establishes the system entry point, binds routes, initializes real-time services, and schedules recurring sensor-status validation.
`backend / src / middleware/auth.ts`	Local authorization	Validates application JWTs and enforces role-based access control for protected routes.
`backend / src / middleware / azureAuth.ts`	Federated identity validation	Verifies Azure AD tokens through issuer, audience, signature, and expiration claims.
`backend / src / routes/auth.ts`	User access and account management	Implements hybrid Azure login, local JWT emission, legacy login, current-user retrieval, and administrative user CRUD.

Module	Primary concern	Methodological role
`backend/src/routes/sensor.ts`	Telemetry ingestion and device control	Receives sensor status, GPS, spectrum, audio, active configuration queries, campaign retrieval, and monitoring commands.
`backend/src/routes/management.ts`	Inventory and alert management	Handles sensors, antennas, assignments, and alert history retrieval.
`backend/src/routes/campaign.ts`	Campaign orchestration	Implements statistics, CRUD, status transitions, data retrieval, and schedule-driven campaign automation.
`backend/src/routes/reports.ts`	Compliance workflow orchestration	Integrates cache policy, GPS resolution, geolocation service calls, and Python post-processing invocation.
`backend/src/routes/config.ts`	System-level tunables	Manages configuration parameters used by monitoring tolerance and timeout logic.
`backend/src/models/Sensor.ts`	Sensor lifecycle policy	Encapsulates sensor identity, state normalization, and status-transition validation.
`backend/src/models/SensorData.ts`	Telemetry persistence and alert thresholds	Persists status, GPS, spectrum, active configuration, in-memory latest data cache, and threshold-based alert creation.
`backend/src/database/connection.ts`	Database connectivity	Defines the PostgreSQL pool, query helpers, and slow-query logging.
`backend/src/database/migrate-postgres.ts`	Schema construction	Creates relational entities, indexes, system defaults, and optional TimescaleDB optimization.
`backend/src/websocket.ts`	General event distribution	Broadcasts status, GPS, spectrum-related events, and generalized client subscriptions.
`backend/src/audioServer.ts`	Audio stream specialization	Accepts sensor audio streams, decodes Opus frames, and redistributes PCM frames to listeners.

4.5 Persistence Model and Data Governance

The persistence strategy is hybrid. Normalized relational tables are used for stable domain entities such as sensors, antennas, users, and campaigns. Semi-structured JSONB is used where configuration variability is expected, notably in campaign configuration and cached compliance reports. Telemetry tables preserve the chronological trace of the system state, while the backend supplements persistence with an in-memory cache for recent sensor spectrum data to support fast monitoring retrieval.

This design is methodologically justified because the platform must support both transactional

integrity and configuration flexibility. However, the evidence also shows an evolutionary schema with mixed temporal representations: some tables store epoch milliseconds in `bigint`, whereas others use PostgreSQL timestamps. This mixture does not invalidate the architecture, but it is a relevant foundation characteristic because all time-sensitive algorithms must compensate for the representational boundary.

4.6 Backend Design Choices and Their Justification

4.6.1 Backend as Source of Truth for Monitoring State

The monitoring lifecycle is persisted in `sensor_configurations` and reinforced by scheduled checks in `sensor.ts`. This is a strong architectural decision. Rather than trusting browser memory, the backend uses persisted active configurations and the `is\_monitoring` flag to decide whether an acquisition should continue or be auto-stopped. The methodological justification is robustness: browser tabs are ephemeral, whereas the backend must preserve operational continuity.

4.6.2 Hybrid Identity Model

The authentication subsystem combines institutional identity validation (Azure AD) with a local application JWT. This allows the platform to trust organizational credentials while still maintaining an application-specific authorization envelope, user roles, and session lifetime. The strategy also enables automatic user provisioning for allowed email domains, which reduces administrative friction.

4.6.3 Route Specialization by Domain Capability

The route decomposition is capability-oriented: authentication, sensor ingestion/control, campaign orchestration, reports, and configuration are each assigned dedicated modules. This is methodologically preferable to a monolithic route file because it preserves domain traceability even though some individual files remain large.

4.6.4 Temporal Automation Inside the Application Layer

The backend contains explicit interval-based automation for sensor state validation and campaign state transitions. This is justified because both problems depend on domain-specific timing rules that are easier to reason about in application code than through ad hoc manual intervention.

4.6.5 Separated Audio Channel

Audio streaming is implemented through a dedicated WebSocket upgrade path and an Opus decoding pipeline distinct from the generic event WebSocket. This specialization is methodologically sound because audio imposes different framing, bandwidth, and listener coordination requirements than simple JSON event traffic.

4.7 Backend Strategy Justification

Strategy	Observed implementation	Justification
Database-first monitoring governance	Active configurations and monitoring flags are stored in `sensor\_configurations`; auto-stop reads from the database.	Prevents loss of control when clients disconnect and gives the backend authoritative operational memory.
Hybrid federated-local authentication	Azure token validation is followed by local JWT issuance.	Balances institutional identity trust with platform-specific role enforcement and session management.
Public ingestion, protected administration	Sensor-facing endpoints are open for device posting, while many management routes require JWTs.	Matches the asymmetric nature of machine-originated telemetry versus human administrative actions, even though enforcement remains heterogeneous across the codebase.
Time-driven campaign orchestration	Campaign states are transitioned using date/time comparisons in Bogotá local time.	Encodes domain time semantics directly and aligns scheduled measurement with operational calendars.
Cache-aware report generation	Cached compliance reports are reused when threshold conditions match; otherwise regeneration is forced.	Reduces repeated expensive analysis while preserving threshold-sensitive analytical correctness.

4.8 Backend Tool Justification

Tool	Observed usage	Justification
Express	Main HTTP server and route composition	Appropriate for a coordination-centric backend that combines many HTTP integration points with middleware chaining.
TypeScript	Route, model, and middleware implementation	Adds interface clarity to a domain with many payload shapes and reduces accidental misuse of configuration envelopes.
PostgreSQL	Primary persistence backend	Supports transactional campaign updates, relational integrity, time-indexed queries, and JSONB-based flexible configuration storage.
TimescaleDB (optional)	Attempted hypertable optimization in migration	Suitable for high-volume time-series telemetry and indicates an intended scale-aware evolution path.
Swagger	API documentation exposure	Improves discoverability and operational testing of a relatively large API surface.
WebSocket (`ws`)	Event broadcast and audio transport	Provides state-change immediacy without requiring client polling for all classes of data.

Tool	Observed usage	Justification
JOSE	Azure token verification	Correctly matches modern OIDC/JWKS validation requirements.
JWT and bcrypt	Local sessions and password handling	Provide standard mechanisms for application-level session control and credential storage where legacy or fallback login is needed.
Axios	Service-to-service HTTP integration	Used in report orchestration to call geolocation and Python analytical services with explicit timeout control.

4.9 Backend Algorithm Justification

Algorithm or rule	Where it appears	Justification
Sensor status decay state machine	`SensorModel.validateAndUpdateStatus()`	Uses recent heartbeat presence to classify sensors into *online*, *delay*, or *offline*; operationally justified for machine availability monitoring.
Threshold-based alert generation with dedup windows	`SensorDataModel.saveStatus()`	Converts raw telemetry metrics into actionable historical alerts while preventing repeated alert flooding.
Database-driven monitoring timeout	`autoStopExpiredMonitoring()` in `sensor.ts`	Ensures bounded monitoring sessions and backend-enforced recovery even after UI loss.
Campaign conflict detection	`campaign.ts` on campaign creation	Prevents one sensor from being scheduled simultaneously in overlapping campaigns, which preserves resource exclusivity.
Timezone-aware campaign status transitions	`updateCampaignStatuses()` in `campaign.ts`	Encodes domain scheduling in the operational timezone rather than relying on ambiguous client-side local clocks.
Center-frequency reconstruction	`sensor.ts` campaign and realtime endpoints	Reconstructs sensor-consumable center frequency from band edges or configuration data, enabling compatibility across evolving payload formats.
Report cache equivalence by threshold	`reports.ts`	Reuses expensive analytical outputs only when the requested threshold is semantically compatible with the cached threshold.
GPS source prioritization	`reports.ts`	Searches manual campaign GPS, fixed sensor coordinates, and then GPS history; this improves reporting resilience when one location source is missing.

Algorithm or rule	Where it appears	Justification
Opus frame parsing and PCM reconstruction	`audioServer.ts`	Specializes audio transport to fit real-time demodulated streams while serving browser listeners with decodable PCM frames.

5 Cross-Cutting Architectural Interpretation

5.1 Observed Strengths

From a methodological standpoint, the current platform exhibits five major strengths.

- **Clear operational decomposition**: monitoring, campaigns, alerts, administration, and reporting are recognizable both in documentation and in code.
- **Strong control-plane orientation**: sensor configuration and campaign lifecycle management are treated as first-class backend concerns rather than incidental UI actions.
- **Pragmatic temporal reasoning**: time-sensitive behaviors are explicitly implemented through validation windows, schedule transitions, and monitoring timeouts.
- **Traceable implementation**: most key concerns can be mapped to named files, making architectural recovery feasible and future governance practical.
- **Evolutionary compatibility**: several modules preserve compatibility with older payload formats while supporting newer configuration structures, reducing disruption across system transitions.

5.2 Observed Constraints and Design Tensions

The repository also reveals design tensions that are relevant for methodological interpretation.

- **Large orchestration modules**: `frontend/src/App.tsx`, `backend/src/routes/sensor.ts`, `backend/src/routes/campaign.ts`, and `backend/src/routes/reports.ts` concentrate significant logic. This improves short-term visibility but raises long-term maintainability pressure.
- **Mixed temporal representations**: the coexistence of epoch-millisecond fields and SQL timestamp fields introduces conceptual friction and increases the burden on scheduling and reporting logic.
- **Evolving time-series strategy**: the legacy database context and current migration file point to different hypertable targets, which suggests ongoing architectural transition rather than final stabilization.
- **Security enforcement heterogeneity**: authentication is explicit and technically competent, but access control application is not uniform across every management route. From a methodological perspective, this means the security design is present, yet not completely normalized in implementation.

- **Boundary dependence on external services**: compliance reporting depends on geolocation and Python analytical services. This is architecturally reasonable, but it makes the backend an orchestration node whose full behavior depends on external availability.

6 Synthesis of Justifications

6.1 Justification of Strategies

The strategies embedded in the codebase are justified because they align with the problem structure of spectral monitoring systems. A backend-centered control plane is necessary because sensors and campaigns persist beyond the life of any single browser session. A hybrid real-time model is necessary because different payload classes impose different transport requirements. Explicit scheduling and timeout rules are necessary because the domain is time-governed rather than merely CRUD-oriented. Finally, report caching is necessary because compliance analysis is computationally and operationally more expensive than simple retrieval.

6.2 Justification of Tools

The chosen tools reflect a preference for mature, inspectable, and integration-friendly technologies. React and TypeScript are suitable where human workflows, data density, and state coordination coexist. Express and PostgreSQL are suitable where orchestration, persistence, and integration dominate over purely computational workload. Swagger, JOSE, Axios, and WebSocket support solve enabling problems rather than domain problems, but they do so in a way that keeps the codebase legible and operationally testable.

6.3 Justification of Algorithms

The implemented algorithms are largely deterministic operational rules rather than mathematically opaque models. This is appropriate for regulatory and monitoring environments because explainability matters. Sensor status transitions, alert thresholds, campaign conflict checks, threshold-equivalent cache reuse, stale-response rejection, and GPS source prioritization can all be audited, reasoned about, and modified without retraining or hidden statistical behavior. In methodological terms, the platform prefers explicit procedural knowledge over black-box inference, which is a sound choice for a supervision and compliance context.

7 Conclusion

The frontend and backend of ANE Spectral Sensing Platformform a coherent operational platform whose primary architectural role is coordination. The frontend is not only a visualization layer; it is a workflow shell that connects authentication, monitoring, campaigns, alerts, and reporting into a continuous operator experience. The backend is not only a CRUD API; it is a domain control plane that persists configurations, mediates sensor actions, enforces timing rules, and orchestrates external analytical services.

From a methodological perspective, the codebase demonstrates a pragmatic and evidence-traceable design. Its strongest characteristics are explicit domain decomposition, temporal control logic, and operational alignment between user workflows and backend authority. Its

main tensions lie in file size concentration, mixed temporal representations, and partial normalization of security enforcement. None of these tensions invalidate the architecture; rather, they define the next areas where the platform could mature while preserving its current strengths.

Accordingly, the present report supports the conclusion that the repository already contains a technically defensible platform foundation for spectral sensing, live monitoring, campaign management, and compliance-oriented reporting. The document also establishes a formal basis for future maintenance, refactoring, or institutional reporting because it links implementation modules to system concerns, strategies, tools, and algorithms in an explicit methodological framework.

A Primary Source Corpus

- `context/CURRENT_PLAT.md`
- `context/API-DOCUMENTATION-BACKEND.md`
- `context/DIAGRAM-BACKEND.md`
- `context/DIAGRAM-FRONTEND.md`
- `context/EDGE_NODE_PROCESSING.md`
- `context/PLAT_PROCESSING.md`
- `context/db/README.md`
- `context/db/inventory.md`
- `context/db/tables.md`
- `context/db/constraints.md`
- `context/db/hypertables.md`
- `context/db/diagram.md`
- Selected implementation files under `frontend/src/` and `backend/src/`

B Representative Implementation Evidence

- Frontend bootstrap and auth: `frontend/src/main.tsx`, `frontend/src/contexts/AuthContext.tsx`
- Frontend orchestration and live monitoring: `frontend/src/App.tsx`, `frontend/src/hooks/useSpectrumData.ts`, `frontend/src/components/ConfigurationPanel.tsx`, `frontend/src/components/AnalysisPanel.tsx`
- Frontend operational workflows: `frontend/src/components/MonitoringNetwork.tsx`, `frontend/src/components/CampaignsList.tsx`, `frontend/src/components/AlertsPanel.tsx`, `frontend/src/components/ComplianceReport.tsx`
- Backend composition and middleware: `backend/src/app.ts`, `backend/src/middleware/auth.ts`, `backend/src/middleware/azureAuth.ts`

- Backend domain orchestration: `backend/src/routes/sensor.ts`, `backend/src/routes/campaign.ts`, `backend/src/routes/reports.ts`, `backend/src/routes/auth.ts`, `backend/src/routes/management.ts`, `backend/src/routes/config.ts`
- Backend persistence and transport: `backend/src/models/Sensor.ts`, `backend/src/models/SensorData.ts`, `backend/src/database/connection.ts`, `backend/src/database/migrate-postgres.ts`, `backend/src/websocket.ts`, `backend/src/audioServer.ts`